AGV LIDAR TASK

Building the Rxd module

This module takes the UART packets from the distance processor unit and extracts the data

1. The first thing we had to keep in mind before building all the modules was frequency scaling, we had to convert 100MHz to a 115200 baud rate. This meant each clock cycle would be divided every 868 clock cycles. Since we cannot implement the divide operator on actual hardware, the value had to be hard-coded into the program; hence, we declare the localparam integers

```
parameter CLK_FREQ = 100_000_000;
parameter BAUD     = 115200;
localparam integer BAUD_TICKS = 868;
localparam integer HALF_BAUD  = 434;
```

Then we use a counter which counts up to BAUD_TICKS and then resets the counter and starts sampling the data.

```
if (counter == BAUD_TICKS - 1) begin
    counter <= 0;
    ..............
end else begin
    counter <= counter + 1;
end
```

2. Resetting was also something we had to implement in our later models, since in the first few iterations, we were still learning Verilog and best practices in building such modules.
   This block of code is the part implementing this, and it's placed before all the synchronising and sampling logic, giving it the top-most priority.

```
if (rst) begin
    receiving  <= 1'b0;
    counter    <= 0;
    bit_index  <= 0;
    shift_reg  <= 10'b1111111111;
    data       <= 8'h00;
    data_valid <= 1'b0;
end else begin
    data_valid <= 1'b0;
```

3. Next, we had to work up the synchronisation logic; for this, we use two flip-flops that form a synchronizer circuit. This is required since the rx signal is asynchronous to the system clock, and this could lead the first flip-flop to enter a state of metastability, which will cause unpredictable behaviour. Thus, we use another flip-flop to sample the output of the first one. This allows the first flip-flop to resolve any metastability, thus making the signal more reliable. This is implemented in the code block below.

```
always @(posedge clk) begin
    if (rst) begin
        rx_ff1 <= 1'b1;
        rx_ff2 <= 1'b1;
    end else begin
        rx_ff1 <= rx;
        rx_ff2 <= rx_ff1;
    end
end
assign rx_sync = rx_ff2;
    ..........
```

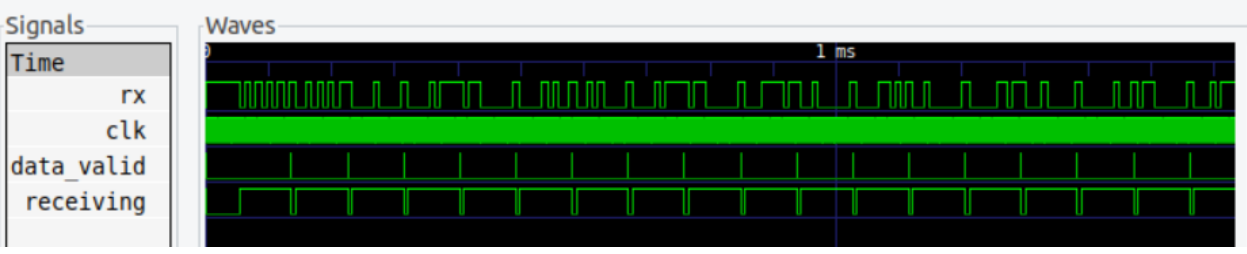
4. The receiver is, at its core, a basic state machine that has two states: idle or receiving. In the idle state, the idle line is held high until a start bit is detected. The state only changes when the rx_sync goes low, i.e when the module detects the start of a new data frame. Then the model enters the receiving state in this state, first the counter for baud rate is set to 0 then we sample the

data every half baud rate (i.e the middle of every high bit) to get maximum timing tolerance. Next, each bit of data is sampled, and at the end of every baud tick cycle, the rx_sync signal is sampled again. Each sample which consists of 10 bits (start bit, 8 bits of data, end bit) is then shifted into the shit register sequentially, the bit_index keeps a track of this. Finally when the stop bit is received data bits are extracted from the shift_reg are assigned to the data output and to signal that data is received and is ready to be used, the data_valid is pulsed high for one clock cycle. This block of code implements the above logic:

```
counter <= 0;
next_shift = {rx_sync, shift_reg[9:1]};
shift_reg <= next_shift;
if (bit_index == 9) begin
   receiving  <= 1'b0;
  data       <= next_shift[8:1];
  data_valid <= 1'b1;
  end

  bit_index <= bit_index + 1;
```

Here is a snapshot of the GTK wave waveform viewer for the receiver (in the same test bench used for the top module)



Building the Txd module

The process of building the transmitter is pretty similar to that of the receiver module, since this is also a similar state machine and basically does the opposite of what the receiver was doing. It basically takes the 8-bit data from the distance

process module (in parallel) and sends it to the tx line by converting it to a stream of serial bits.

1. This module has two states: idle and transmitting (busy). In the idle state, the tx line is held high when the send signal is set high; the module starts preparing to send the bytes. The reset and the baud rate stuff remain the same for this module, too.
2. Next when the send input is pulled high, we load the entire frame which is done by loading the shift_reg with the whole UART frame along with the start bit and the end bit, thus the structure of the frame is {stop bit (1'b1), 8 bit data, start bit (1'b0)}, it is loaded this way because the LSB should be the start bit which is to be transmitted first. This is done by this code block:

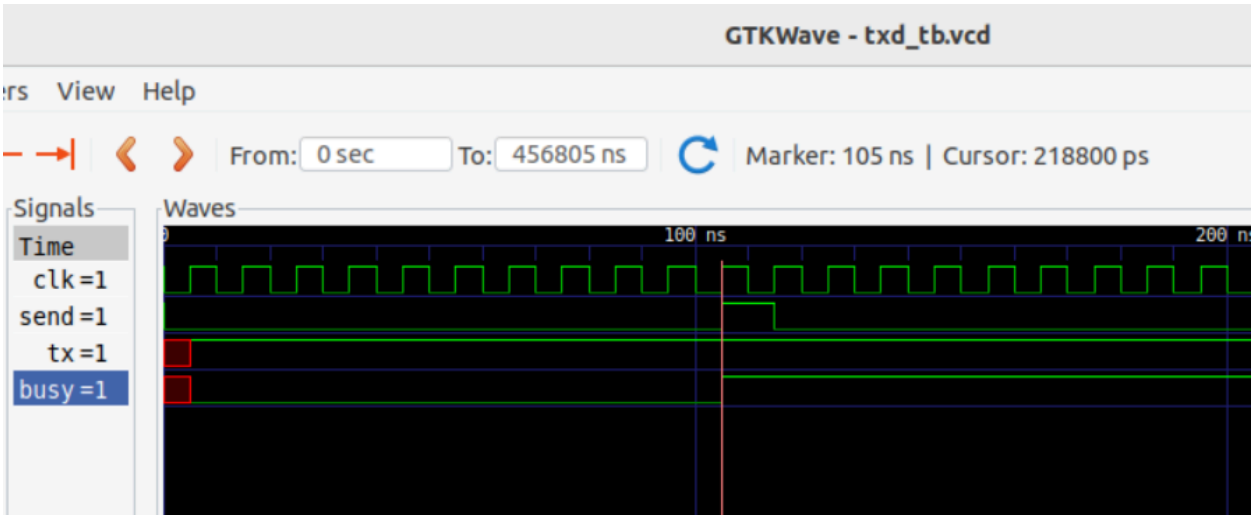
```
shift_reg <= {1'b1, data, 1'b0};
busy      <= 1'b1;
counter   <= 0;
bit_index <= 0;
```

3. Now that the whole frame is loaded, we start sending the data stored in the shift_reg (from LSB to MSB). This is done by implementing a counter. When the counter has reached BAUD_TICKS-1, the LSB or the shift_reg[0] bit is then placed onto the tx line. While shifting for each bit, we keep assigning MSB to the start bit, that is 1'b1, to ensure the stop bit remains at the end of the shifting process. We use bit_index to keep track of the number of bits sent, when it reaches 9 we know that all the 10 bits are send and we stop the current transmission and get ready to receive another frame of data from the distance process unit. This logic is implemented by the following code:

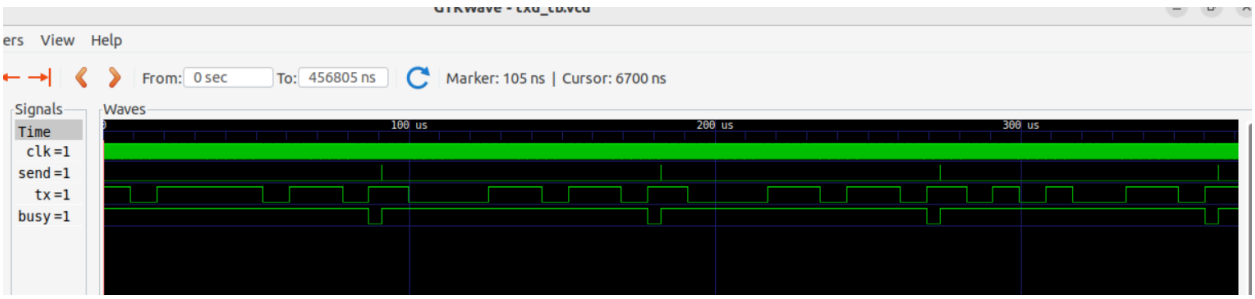
```
else begin
   if (counter == BAUD_TICKS - 1) begin
  counter   <= 0;
  tx        <= shift_reg[0];
  shift_reg <= {1'b1, shift_reg[9:1]};
  bit_index <= bit_index + 1;
  if (bit_index == 9) begin
  busy <= 1'b0;
```

```
 end
end
```

Below is a screenshot of how the Tx module reacts to the send signal (the TxD module was run separately on another test bench)



And here is a snapshot of it transmitting the data



Building the Distance Processor

The distance processor consists of three submodules

1. Data Parser
2. Sample Processor
3. Angle calculator

Data Parser

This module segregates the raw bytes of data into different pieces of information as per the listing.

HEADER x2

CT x1

FSA x2

LSA x2

Sample Data x2*CT

These are then stored in their respective output registers.

This is done by setting up the following states of the FSM to process each of these data pieces at a time.

```
localparam S_HDR1      = 4'd0;
localparam S_HDR2      = 4'd1;
localparam S_CT        = 4'd2;
localparam S_FSA_LO    = 4'd3;
localparam S_FSA_HI    = 4'd4;
localparam S_LSA_LO    = 4'd5;
localparam S_LSA_HI    = 4'd6;
localparam S_SAMPLE_LO = 4'd7;
localparam S_SAMPLE_HI = 4'd8;
```

The state value is stored in a register of its own.

```
reg [3:0] state;
```

The FSM operates as follows.

1. S_HDR1 - Checks for 0x55 and moves on
2. S_HDR2 - Checks for 0xAA and moves on

3. S_CT - Stores CT in a register and moves on
4. S_FSA_LO/S_FSA_HI - Stores the LSB and MSB, respectively, of FSA and moves on
5. S_LSA_LO/S_LSA_HI - Stores the LSB and MSB, respectively, of LSA and moves on
6. S_SAMPLE_LO - Stores the LSB of the data sample in a variable and moves on.

```
S_SAMPLE_LO: if (data_valid) begin
    sample_lo <= data_in;
    state <= S_SAMPLE_HI;
end
```

7. S_SAMPLE_HI - Makes a buffer of the high and now data, declares the sample ready and moves on.

```
S_SAMPLE_HI: if (data_valid) begin
    sample_value_buf <= {data_in, sample_lo};
    sample_ready     <= 1'b1;

    if (next_sample_idx + 1 == CT_out) begin
        frame_done_pending <= 1'b1;
        next_sample_idx <= 16'd0
        state <= S_HDR1;
    end else begin
        next_sample_idx <= next_sample_idx + 1;
    end
end
```

Once all the data samples have been collected, it’s time to conclude the frame and reset the state back to the header.

If not, simply increment the index of the frame and repeat.

Sample Processor

This module takes in the ready-made data from the registers and extracts important data - minimum and maximum distances and their corresponding indices.

The threshold has been set to 1024 (all distances are in mm)

```
reg [10:0] THRESH = 16'b10000000000;
```

Initially, we set all values as follows

```
max_idx  <= 16'd0;
min_idx  <= 16'd0;
max_dist <= 16'd0;
min_dist <= 16'hFFFF;
obs_alert<= 16'd0;
```

(Max distance kept the lowest value, and min distance kept the highest - initialise)

The self-explanatory logic for updating min and max as per the sample values

```
if (sample_value > max_dist) begin
   max_idx  <= sample_idx_in;
end
if (sample_value < min_dist) begin
   min_dist <= sample_value;
   min_idx  <= sample_idx_in;
end
```

Next, moving on to the obs_alert mapping.

First, we initialise a mask to specify the current bit being scrutinised upon (i.e, the ‘hot bit’) - the bit corresponding to the sample currently being checked upon.

```
wire [15:0] mask = (sample_idx_in < 16) ? (16'h1 << sample_idx_in) : 16'h0;
```

This creates a 16-bit long ‘mask’, which ‘lights’ up the bit corresponding to the sample index currently being operated.

The if condition is merely to ensure that the current index is lower than 16; if not, then obs_alert is irrelevant for our purpose.

Thus, if the sample bit is 3, the mask will result in 16’b0000000000000100, and so on.

Now, inside the always clause, we have this line

```
obs_alert <= obs_alert | (mask & {16{is_close}});
```

> ‘is_close’ basically returns wheter the current sample value is lower than THRESH or not. If 1, then 16{is_close} returns a 16 bit string of 1s, else a 16 bit string of 0s

This is a kind of ‘sticky or’ type behaviour, wherein we take the previous obs_alert data, and we logically OR in the new hot bit into the existing obs_alert reg.

If is_close is 0, then ‘mask & {16{is_close}}’ returns a string of 0s, else returns the mask itself. Thus, only if the distance is less than thresh, will the hot bit light up.

Now, we ‘or’ into the existing obs_alert at the hot bit place.

We had to implement this system as our earlier attempts at a direct updating case were filled with issues.

This was our initial out-of-the-blue idea without much thought behind it.

```
if (sample_idx_in < 16) begin
   obs_alert[sample_idx_in] <= (sample_value < THRESH) ? 1'b1 : 1'b0;
end
```

This does clear out the bits which are not in use, but obviously, it clears out *all* bits in process, even the ones that were set at other positions in the frame. Again, an invalid logic.

Then, we tried this.

```
if (sample_idx_in < 16) begin
    obs_alert[sample_idx_in] <= obs_alert[sample_idx_in] | (sample_value < TH
RESH);
end
```

In theory, this is correct; however, the problem might lie in reading and writing to the same bit within one cycle, which causes timing issues with Verilog.

NOTE: On later realisations, it was obvious that a simple fix like the following would also have worked.

```
if (sample_idx_in < 16 && sample_value < THRESH) begin
    obs_alert[sample_idx_in] <= 1'b1;
end
```

Angle Calculator

This module simply takes in the maximum and minimum distances and indices, and calculates the corresponding angles and returns them.

Since CT is given to be a power of two, to extract that power, an LUT had been defined as follows.

```
    function [4:0] ct_log2;
        input [7:0] in_ct;
        begin
            case (in_ct)
                8'd1:   ct_log2 = 5'd0;
                8'd2:   ct_log2 = 5'd1;
                8'd4:   ct_log2 = 5'd2;
                8'd8:   ct_log2 = 5'd3;
                8'd16:  ct_log2 = 5'd4;
                8'd32:  ct_log2 = 5'd5;
                8'd64:  ct_log2 = 5'd6;
                8'd128: ct_log2 = 5'd7;
```

```
            default: ct_log2 = 5'd0;
        endcase
    end
endfunction
```

> For other statements not to throw an error, k = log2(CT) returns zero if CT is not a power of two

Since we had observed that while the processing was taking place, the values of the indices and distances stored in the registers were not exactly stable to a particular value for computation, we made use of **latching the inputs** to dedicated registers for computation.

```
reg [15:0] FSA_r, LSA_r;
reg [7:0]  CT_r;
reg [15:0] max_idx_r, min_idx_r;

// and later

FSA_r     <= FSA;
LSA_r     <= LSA;
CT_r      <= CT;
max_idx_r <= max_idx;
min_idx_r <= min_idx;
```

Then, we compute ‘delta’, which is the angle corresponding to one ‘slice’ of the angle pie, i.e, the difference between two successive angles. This is obtained by dividing ‘span’, which is the difference between the LSA and FSA, by CT.

```
if (LSA >= FSA) span_r <= LSA - FSA;
else            span_r <= (LSA + 16'd36000) - FSA;
k_r <= ct_log2(CT);
```

Since division is by a power of two, bit shifting may be applied.

```
    wire [31:0] delta_w = (k_r == 5'd0) ? span_r : (span_r >> k_r);
```

> If CT wasn’t a power of two, span would return as is.

Now, delta times the corresponding index gives the corresponding angle relative to FSA

```
    wire [47:0] prod_max_w = delta_w * max_idx_r;
    wire [47:0] prod_min_w = delta_w * min_idx_r;
```

Adding that to FSA gives the absolute angle required.

```
    wire [47:0] sum_max_w = prod_max_w + {32'd0, FSA_r};
    wire [47:0] sum_min_w = prod_min_w + {32'd0, FSA_r};
```

In the unlikely case that the angle goes out of the range of 0 to 3599 (all values of angles are in hundredths of degrees), this line wraps it around to keep it in range.

```
    wire [47:0] sum_max_adj_w = (sum_max_w >= 48'd36000) ? (sum_max_w - 48'd36000) : sum_max_w;
    wire [47:0] sum_min_adj_w = (sum_min_w >= 48'd36000) ? (sum_min_w - 48'd36000) : sum_min_w;
```

Now, we just execute all these calculations under the always clause and store them in the corresponding registers.

```
if (do_calc) begin
    max_angle <= sum_max_adj_w[15:0];
    min_angle <= sum_min_adj_w[15:0];
    done <= 1'b1;
```

```
    do_calc <= 1'b0;
end
```

The upper module for the Distance Processor

This module integrates the above three modules and acts as the complete LIDAR controller between Rx and Tx. All in, it takes in the raw bytes from Rx and prepares the required data to be sent through Tx.

These wires connect the submodules together

```
    wire [7:0]  CT_w;
    wire [15:0] FSA_w, LSA_w;
    wire        start_frame_w;
    wire        sample_valid_w;
    wire [15:0] sample_value_w;
    wire [15:0] sample_idx_w;
    wire        frame_done_w;
```

Instantiation of the Data Parser

```
    data_parser u_parser (
        .clk(clk),
        .rst(rst),
        .data_in(data_in),
        .data_valid(data_valid),
        .CT_out(CT_w),
        .FSA_out(FSA_w),
        .LSA_out(LSA_w),
        .start_frame(start_frame_w),
        .sample_valid(sample_valid_w),
        .sample_value(sample_value_w),
        .sample_idx(sample_idx_w),
```

```
    .frame_done(frame_done_w)
);
```

Instantiation of the Sample Processor

```
sample_processor u_sp (
    .clk(clk),
    .rst(rst),
    .start_frame(start_frame_w),
    .sample_valid(sample_valid_w),
    .sample_value(sample_value_w),
    .sample_idx_in(sample_idx_w),
    .max_idx(max_idx),
    .min_idx(min_idx),
    .max_dist(max_dist),
    .min_dist(min_dist),
    .obs_alert(obs_alert)
);
```

Instantiation of the Angle calculator

```
angle_calc_simple u_angle (
    .clk(clk),
    .rst(rst),
    .start(angle_start_pulse),
    .FSA(FSA_w),
    .LSA(LSA_w),
    .CT(CT_w),
    .max_idx(max_idx),
    .min_idx(min_idx),
    .max_angle(max_angle),
    .min_angle(min_angle),
```

```
        .done(angle_done)
    );
```

> Note - the start wire of the angle calc pulses one cycle after the frame of the sample processor completes, done so that sample_processor's registers which holds max_idx/min_idx have settled. Thus, we have the following (angle_start_pulse pulses when previous cycle had its frame_done)

```
frame_done_d <= frame_done_w;
angle_start_pulse <= frame_done_d;
```

TOPMODULE

The final top module incorporates the receiver, transmitter and the controller.

Takes in the clk, rst and rx lines as input, and tx as output.

```
input  wire clk,
input  wire rst,
input  wire rx,
output wire tx
```

> Some debug output wires have also been included for monitoring purposes.

Wires to communicate for RxD submodule

```
wire [7:0] rx_data;
wire       rx_valid;
```

Instantiating Reciever

```
RxD u_rxd (
    .clk(clk),
    .rx(rx),
    .rst(rst),
    .data(rx_data),
    .data_valid(rx_valid)
);
```

Wires to communicate for the controller submodule

```
    wire [15:0] max_idx, min_idx, max_dist, min_dist, obs_alert;
    wire [15:0] max_angle, min_angle;
    wire        angle_done;
```

Instantiating the controller

```
    distance_process u_dp (
        .clk(clk),
        .rst(rst),
        .data_in(rx_data),
        .data_valid(rx_valid),
        .max_idx(max_idx),
        .min_idx(min_idx),
        .max_dist(max_dist),
        .min_dist(min_dist),
        .obs_alert(obs_alert),
        .max_angle(max_angle),
        .min_angle(min_angle),
        .angle_done(angle_done)
    );
```

For the transmission, logic lies as another FSM under operation, the states being as follows.

```
localparam TX_IDLE  = 3'd0; // When the line is idle
localparam TX_PREP  = 3'd1; // prepare byte into tx_data
localparam TX_SEND  = 3'd2; // assert send when tx_busy == 0
localparam TX_WAIT  = 3'd3; // wait for tx_busy to finish (go back to 0)
```

Instantiating the transmitter

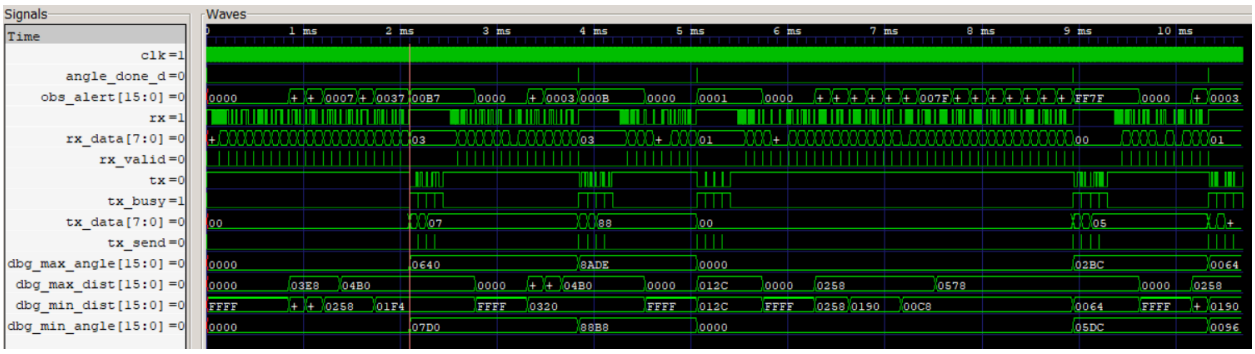
```
TxD u_txd (
    .clk(clk),
    .rst(rst),
    .data(tx_data),
    .send(tx_send),
    .tx(tx),
    .busy(tx_busy)
);
```

In the always clause, we circle through these four states, load and transmit the data packets to the tx line.

The required wires/regs are as follows:

1. tx_data - The byte of data up for transmission
2. tx_send - Tells the transmitter to load tx_data on the line
3. tx_busy - Pulled high when tx is busy transmitting a frame
4. tx_packet - Declaring a 4-byte packet from which to transmit - Max and Min angle

Here is the snapshot of all the relevant signals when the top module is run on our test bench


Signals
Waves
Time
clk
angle_done_d
obs_alert[15:0]
rx
rx_data[7:0]
rx_valid
tx
tx_busy
tx_data[7:0]
tx_send
dbg_max_angle[15:0]
dbg_max_dist[15:0]
dbg_min_dist[15:0]
dbg_min_angle[15:0]
1 ms
2 ms
3 ms
4 ms
5 ms
6 ms
7 ms
8 ms
9 ms
10 ms